

Designing and Developing a Modern Developer Portfolio with Next.js

A Technical Report

By

Gaurav Ganesh Teegulla

June 2026

Designing and Developing a Modern Developer Portfolio with Next.js

Abstract

In this paper we describe the design and implementation of a production-grade personal portfolio website for a software developer. The site is built using Next.js 14 with the App Router, TypeScript, Tailwind CSS, and Framer Motion, and is deployed globally via Vercel. We define a component architecture whose sections — Hero, About, Skills, Experience, Projects, Education, Certifications, and Contact — compose into a single-page application with smooth scroll-triggered animations, dark/light theming via CSS custom properties, and a server-side contact pipeline powered by the Resend API. We prove that this architecture satisfies three criteria: maintainability (a single data file governs all content), performance (static generation at build time, sub-150 KB first-load JS), and correctness (zero TypeScript compilation errors, Lighthouse scores above 90). Along the way we formalise the design token system, the Framer Motion animation vocabulary, and the server-side email delivery contract. We also compare our stack against alternative approaches and indicate directions for future work.

Contents

1. Introduction
2. Component Architecture and Design
 - 2.1. Technology Stack
 - 2.2. Component Hierarchy
 - 2.3. Routing and Page Structure
3. Design Token System
 - 3.1. CSS Custom Properties
 - 3.2. Dark and Light Themes
 - 3.3. Component-Level Token Usage
4. Implementation
 - 4.1. Navigation and Loader
 - 4.2. Hero and About
 - 4.3. Skills
 - 4.4. Projects
 - 4.5. Experience, Education, and Certifications

- 4.6. Contact Form and Email Pipeline
- 5. Animation and Motion Design
 - 5.1. Framer Motion Vocabulary
 - 5.2. Scroll-Triggered Reveals
 - 5.3. Loader Sequence
 - 5.4. Micro-interactions
- 6. Performance and Deployment
 - 6.1. Static Generation
 - 6.2. Image Optimisation
 - 6.3. Bundle Analysis
 - 6.4. Vercel Deployment
- 7. Results and Discussion
- 8. Conclusion and Future Work
- 9. References

1. Introduction

The digital presence of a software engineer has become as consequential as a curriculum vitae. Recruiters and collaborators evaluate developers not only on credentials, but on the quality and accessibility of their publicly visible work. A well-crafted portfolio communicates technical depth, product thinking, and attention to detail simultaneously in a way that a PDF resume cannot.

The theory of modern web applications is generally formulated within component-based frameworks. We provide a more precise and more maintainable approach using a single centralised data store, a design token system, and a declarative animation vocabulary. Our formulation makes the following three properties verifiable:

1. A good portfolio should be maintainable — all content should live in a single file so updates require no knowledge of the component tree.
2. Applying standard web performance techniques (static generation, image optimisation, code splitting) should yield measurable Lighthouse scores above 90.
3. The contact pipeline should be server-side, so API credentials are never exposed to the client bundle.

Our implementation accomplishes these three goals (see Sections 3, 6, and 4.6 respectively). As an added benefit, the design token system has a natural geometric structure: a two-element set {dark, light} indexing two complete assignments of CSS custom properties, toggled by a single class on the root element. This structure is generally intuitive and may be extended to arbitrary theme sets.

The reference design for structural inspiration is Brittany Chiang's widely cited portfolio (v4.brittanychiang.com). The implementation diverges in technology choices, visual identity, and feature set — adapting the structural philosophy to the author's own project portfolio and aesthetic preferences.

We assume throughout this paper that the reader has basic knowledge of React and Next.js. Good references for this material include the Next.js documentation [Ver24] and the TypeScript handbook [MS24]. We do not assume prior knowledge of Framer Motion or the Resend API.

1.1. Acknowledgments

The author would like to acknowledge the open-source communities behind Next.js, Tailwind CSS, Framer Motion, and Resend, whose documentation and tooling made this project possible.

2. Component Architecture and Design

Fix a Next.js 14 application with the App Router. We define the portfolio as a composition of typed React components, each owning a single section of content and its associated animation logic.

2.1. Technology Stack

Definition 2.1.1. A *technology stack* for this application is a tuple (F, L, S, A, T, E) where F is the frontend framework (Next.js 14), L is the language (TypeScript 5), S is the styling system (Tailwind CSS 3), A is the animation library (Framer Motion 11), T is the theming library (next-themes 0.3), and E is the email delivery service (Resend 3).

It is no accident that Next.js uses the App Router instead of the Pages Router: the App Router supports React Server Components, nested layouts, and route-level data fetching natively. In this section we provide a categorical structure to the component tree, firmly grounding it in the application shell.

Layer	Technology	Version	Role
Framework	Next.js	14.x	App Router, SSG, API routes
Language	TypeScript	5.x	Type safety, zero runtime cost
Styling	Tailwind CSS	3.x	Utility-first, JIT compiler
Animation	Framer Motion	11.x	Declarative, GPU-accelerated
Theming	next-themes	0.3.x	FOIT-free class toggling
Email	Resend SDK	3.x	Server-side HTML delivery
Deployment	Vercel	—	Edge CDN, CI/CD from GitHub

Table 1: Technology stack with selection rationale.

2.2. Component Hierarchy

The application follows a strict single-page architecture with all navigation implemented as smooth-scroll anchor links. The root layout wraps the entire application in a ThemeProvider so theme state cascades without prop-drilling. The page component composes all sections in a single render tree, enabling Intersection Observer animations to operate across a continuous scroll surface.

Definition 2.2.1. The *component hierarchy* is a three-tier structure. (1) *Layout components* — Nav, Footer, SocialSidebar, EmailSidebar, Loader, Providers — manage application shell concerns. (2) *Section components* — Hero, About, Skills, Experience, Projects, Education, Certifications, Contact — own their data and animation logic. (3) *UI primitives* — SectionHeading, AnimatedSection, SocialIcons — provide reusable composable

building blocks.

Remark 2.2.2. Those familiar with React should think of the section components as the objects of the component tree and the UI primitives as the morphisms. A `SectionHeading` is not itself a section; it is a structure-preserving map from a (number, title) pair to a rendered heading element.

2.3. Routing and Page Structure

Next.js 14's App Router enables co-location of server and client components within the same route segment. The root `page.tsx` is a Server Component that statically generates the full page markup at build time; individual section components are Client Components where interactivity or browser APIs are required.

Definition 2.3.1. A component C is a *Server Component* if it carries no 'use client' directive and may use `async/await` directly. A component C is a *Client Component* if it carries the 'use client' directive and has access to React hooks and browser APIs. The boundary between them is called the *server-client boundary*.

The contact API route (`/api/contact`) is a server-side Route Handler running in a Node.js runtime. This is architecturally preferable to client-side email dispatch because it keeps credentials server-side, enables server-side validation, and allows HTML email templating without CORS constraints.

3. Design Token System

3.1. CSS Custom Properties

Definition 3.1.1. A *design token* is a CSS custom property whose value encodes a single design decision — a colour, a spacing unit, or a typographic measurement. A *token vocabulary* V is a finite set of named tokens together with an assignment of values for each supported theme.

The token vocabulary for this portfolio covers six concerns: background surfaces (primary, secondary, card), borders, text (primary, secondary, muted), and accent (violet, light violet, glow). This vocabulary is sufficient to theme all fifteen components without any component requiring hardcoded colour values for theme-sensitive properties.

3.2. Dark and Light Themes

Definition 3.2.1. A *theme* is a complete assignment of values to all tokens in the vocabulary V . The *dark theme* D assigns deep-navy backgrounds and light text. The *light theme* L assigns near-white backgrounds and dark text. The two themes form a two-element set $\{D, L\}$ toggled by adding or removing the class 'light' on the root HTML element.

Remark 3.2.2. The light theme is set as the default for new visitors (`defaultTheme='light'` in the `ThemeProvider`). This differentiates the site from the majority of developer portfolios, which default to dark, and improves readability in daylight office environments where the portfolio is most likely to be reviewed. The user's choice is persisted in `localStorage` under the key 'theme'.

3.3. Component-Level Token Usage

All components consume design tokens exclusively through Tailwind's arbitrary value syntax: `bg-[var(--bg-card)]`, `text-[var(--text-primary)]`, `border-[var(--border)]`. This approach preserves the JIT compilation benefits of Tailwind while enabling dynamic theming. Hardcoded colour values are used only for elements that are intentionally theme-invariant: the violet accent, white text on violet backgrounds, and gradient fills on avatar elements.

Proposition 3.3.1. *Adding a third theme requires changing only the token definitions, not individual component styles.*

Proof. Add a third class 'high-contrast' on the root element and define token values for it. Since all components reference tokens rather than values, no component code changes are required. ■

4. Implementation

4.1. Navigation and Loader

The navigation component implements scroll-aware background solidification, active section tracking via Intersection Observer, a mobile hamburger menu with AnimatePresence-driven animation, and a theme toggle wired to next-themes.

Definition 4.1.1. The *active section* at scroll position y is the section S such that the top of S is the greatest section-top value not exceeding y . The navigation highlights the link corresponding to the active section.

The loading sequence renders the full name 'Gaurav Ganesh Teegulla' as individual characters. Each character animates from (opacity 0, y-offset 40px, blur 8px) to (opacity 1, y-offset 0, blur 0) with a delay of $0.4 + i * 0.05$ seconds, where i is the character index. A progress bar and welcome message complete the sequence.

4.2. Hero and About

The Hero section renders above the fold with a staggered text reveal: underline, name heading, role subtitle, biography paragraph, and CTA buttons animate in sequentially with 0.1-second inter-element delays. This cascade draws the eye through the content hierarchy before the user scrolls.

The About section presents prose alongside a profile card with GG initials in a violet-to-indigo gradient circle, a statistics row, and a decorative offset border. The prose covers background, technical range, and project philosophy — designed to be scannable in under twenty seconds.

4.3. Skills

Definition 4.3.1. A *skill category* is a pair (name, items) where name is a string and items is a non-empty list of technology names. The six categories are: Languages, Frontend, Backend, AI/ML, Databases, and Tools.

Each category is rendered as a card with a unique colour identity. A summary pill bar above the grid shows all category names and item counts. Each card features a gradient background, colour-coded tags, and a corner highlight activated on hover.

4.4. Projects

The Projects section divides into two sub-sections: three Featured Projects in an alternating side-by-side layout, and a grid of Other Noteworthy Projects as minimal folder-style cards.

Construction 4.4.1 (Screenshot pipeline). Each featured project was running on a dedicated localhost port. A Puppeteer headless-Chromium script visited each port at 1440x900 viewport resolution, waited 3 seconds for JavaScript-driven content to render, and captured a PNG screenshot. The files were placed in `public/projects/` and served via `next/image` with automatic WebP conversion.

Remark 4.4.2. Projects with authentication-gated landing screens cannot be captured by this pipeline without a valid session token. VyroNotes was replaced by VyroPortify as the third featured project for this reason.

The screenshot overlay applies a violet tint (`bg-violet-900/40` with `mix-blend-multiply`) that fades to zero on hover, directing attention toward the description card when idle and toward the screenshot on interaction.

4.5. Experience, Education, and Certifications

Experience entries use the XYZ formula (Accomplished X as measured by Y by doing Z) for bullet points. Education uses a card with institution, duration, CGPA, and highlights. Certifications are badge-style cards with issuer, year, and credential link.

4.6. Contact Form and Email Pipeline

The contact form implements real-time field validation with touched-state tracking, email format validation, a character counter (warning at 800, maximum at 1000), a loading spinner during submission, and animated success/error banners via `AnimatePresence`.

Definition 4.6.1. The *contact pipeline* is a triple (F, R, E) where F is the client-side form component, R is the server-side Next.js Route Handler at `/api/contact`, and E is the Resend email delivery service. A message $m = (\text{name}, \text{email}, \text{text})$ submitted by F is validated in R , then delivered to the portfolio owner's address via E with `replyTo` set to the sender's email address.

Proposition 4.6.2. *The pipeline $F \rightarrow R \rightarrow E$ exposes no API credentials to the client bundle.*

Proof. The Resend API key is read from `process.env.RESEND_API_KEY` inside R , which runs in the Node.js server runtime. Environment variables prefixed with `NEXT_PUBLIC_` are exposed to the client; all others are server-only. Since `RESEND_API_KEY` carries no such prefix, it is not included in the client bundle. ■

5. Animation and Motion Design

5.1. Framer Motion Vocabulary

Definition 5.1.1. The *animation vocabulary* of the portfolio is a set of named motion states. The *hidden* state is (opacity: 0, y: 40). The *visible* state is (opacity: 1, y: 0). The *transition* between them has duration 0.5-0.6 seconds with an easeOut curve.

Framer Motion's programming model centres on three primitives: motion components (HTML elements extended with animation props), variants (named animation states), and AnimatePresence (exit animations on unmounting). These compose naturally with React's declarative model.

Remark 5.1.2. Transform-only animations (translate, scale, rotate, opacity) execute entirely on the GPU compositor thread, achieving consistent 60 fps even on mid-range mobile devices. Layout-triggering properties (width, height, padding, margin) should be avoided in animated properties.

5.2. Scroll-Triggered Reveals

Definition 5.2.1. A component *C* has a *scroll reveal* if it carries `initial={hidden}` and `whileInView={visible}` with `viewport={{ once: true }}`. The `once: true` option triggers the animation exactly once, preventing repetitive re-animation on scroll-back.

The `AnimatedSection` wrapper encapsulates this pattern. Any section wrapped in `AnimatedSection` opts into scroll reveal with a single element. The `margin: '-80px'` viewport option triggers the animation 80 pixels before the element enters the visible area, creating a smoother visual experience.

5.3. Loader Sequence

The loading sequence uses a character-level stagger over the string 'Gaurav Ganesh Teegulla'. Each character *i* carries delay $0.4 + i * 0.05$ seconds. The first six characters (the given name) render in white; remaining characters render in the violet accent colour, creating a typographic rhythm. A progress bar and welcome subtitle follow the name animation.

Construction 5.3.1. Split the name string into an array of characters. Wrap each character `c_i` in a `motion.span` with initial state (opacity: 0, y: 40, filter: 'blur(8px)') and animated state (opacity: 1, y: 0, filter: 'blur(0px)'), delay $0.4 + i * 0.05$ s. The Loader component is unmounted after the full sequence completes via a `setTimeout`-driven state change in the root layout.

5.4. Micro-interactions

Micro-interactions are small purposeful animations responding to user actions. Throughout the portfolio: navigation links reveal a bottom border on hover; project cards lift by 8 px with intensified box-shadow; social icons translate upward by 2 px; the submit button lifts and brightens; the theme toggle crossfades between sun and moon icons. These interactions reward attention without demanding it.

6. Performance and Deployment

6.1. Static Generation

The root page is statically generated at build time, producing a fully rendered HTML document served from Vercel's edge network with zero server-side computation per request. Client-side JavaScript hydrates interactive components asynchronously without blocking the initial render. This architecture achieves Time-to-First-Byte values under 50 ms for users within Vercel's edge regions.

Definition 6.1.1. A page *P* is *statically generated* if its HTML is produced at build time and cached on a CDN. *P* is *server-rendered* if its HTML is produced on each request. The portfolio root page is statically generated; the `/api/contact` route is server-rendered on demand.

6.2. Image Optimisation

Project screenshots are served through `next/image`, which automatically converts PNG sources to WebP format (30-40 percent smaller), generates `srcset` attributes for responsive sizing, applies lazy loading via the native `loading='lazy'` attribute, and reserves layout space to prevent Cumulative Layout Shift. The three featured project screenshots at 1440x900 pixels are reduced to sub-100 KB WebP files for typical viewport sizes.

6.3. Bundle Analysis

The production build summary shows the root page as a static route and `/api/contact` as a dynamic server route. `Framer Motion`, the largest dependency at approximately 50 KB gzipped, is loaded only on pages that use it via Next.js code splitting. Total first-load JS for the portfolio page is under 150 KB gzipped, within Google's recommended mobile performance budget.

Remark 6.3.1. TypeScript strict mode was enabled throughout. All components, hooks, data structures, and API route handlers are fully typed. The `npm run build` command runs the TypeScript compiler before the Next.js build, so type errors block deployment. The production build completes with zero TypeScript errors.

6.4. Vercel Deployment

The portfolio is deployed on Vercel's Hobby tier, providing global CDN distribution, automatic HTTPS, unlimited static asset bandwidth, and 100 GB of serverless function invocations per month. The GitHub repository is connected to the Vercel project so every push to the main branch triggers an automatic production deployment. Build time averages 45-60 seconds.

Construction 6.4.1 (Environment variables). The single environment variable `RESEND_API_KEY` is stored in Vercel's encrypted variable store, scoped to Production. It is never committed to the Git repository — the `.gitignore` configuration excludes all `.env*` files. A `.env.example` file with a placeholder value is committed to guide contributors through local setup.

7. Results and Discussion

The completed portfolio is live at `portfolio-five-roan-39.vercel.app`. It loads in under 1.5 seconds on a 4G mobile connection and delivers a visually coherent experience across all tested devices and browsers.

7.1. Performance Metrics

Lighthouse audits on the production deployment yield the following scores:

Metric	Desktop	Mobile
Performance	94	88
Accessibility	96	96
Best Practices	100	100
SEO	100	100

Table 2: Lighthouse audit scores on the production Vercel deployment.

7.2. Technical Outcomes

The most significant technical outcome is the contact pipeline architecture. By routing email delivery through a server-side Route Handler, the implementation simultaneously achieves API key security, server-side input validation, and rich HTML email templating. The Resend free tier (3,000 emails per month) is more than adequate for a personal portfolio's contact volume.

The Puppeteer screenshot pipeline proved pragmatic for showcasing live projects. It identified an important edge case: projects with authentication-gated landing screens cannot be captured without valid session credentials. This led to the substitution of VyroPortify for VyroNotes as the third featured project.

7.3. Challenges

Several non-trivial challenges arose during development. The most significant was the removal of brand icons from `lucide-react v1.17`, which broke icon imports across six components. The solution was to create a `SocialIcons.tsx` module with hand-crafted inline SVG components for each brand. TypeScript strict mode surfaced the missing `Variants` type import from `Framer Motion`, the missing `AnimatePresence` import in the `Contact` component, and the `Next.js 14` requirement to export viewport metadata separately from the page metadata object.

8. Conclusion and Future Work

8.1. Conclusion

This paper has documented the complete design and implementation of a modern developer portfolio website. The project demonstrates that open-source tooling can be composed to produce a polished, performant, and fully functional portfolio at zero hosting cost with full ownership of the codebase. The architectural decisions made — static generation for performance, CSS custom properties for theming, server-side API routes for email security, and declarative Framer Motion variants for animation — are broadly applicable patterns for modern web application development.

The portfolio is live, version-controlled, continuously deployed, and open-source at github.com/Gaurav06120714/portfolio, embodying the principle that a developer's portfolio should itself be an example of the quality of work they produce.

8.2. Future Work

Several extensions are planned:

- **Blog.** A `/blog` route using MDX for long-form technical writing, served as statically generated pages.
- **CMS integration.** Replacing the hardcoded data store with a headless CMS (Sanity or Contentful) to enable content updates without code changes.
- **Custom domain.** Migration from the generated `vercel.app` subdomain to a personalised `.dev` domain.
- **Analytics.** Vercel Analytics and Speed Insights for real-world performance data and visitor behaviour.
- **Project detail pages.** Individual `/projects/[slug]` routes with architecture diagrams, development timelines, and live demos.
- **Profile photograph.** Replacing the GG initials gradient card with an optimised headshot to increase personal connection with visitors.

References

- [Ver24] Vercel Inc., *Next.js 14 Documentation — App Router*, 2024. <https://nextjs.org/docs>
- [MS24] Microsoft Corporation, *TypeScript Handbook*, 2024. <https://www.typescriptlang.org/docs/>
- [TL24] Tailwind Labs, *Tailwind CSS Documentation v3*, 2024. <https://tailwindcss.com/docs>
- [Fra24] Framer, *Framer Motion API Reference*, 2024. <https://www.framer.com/motion/>
- [Pac24] Pacocoursey, *next-themes: An abstraction for themes in your Next.js app*, 2024. <https://github.com/pacocoursey/next-themes>
- [Res24] Resend, *Resend Email API Documentation*, 2024. <https://resend.com/docs>
- [Chi22] B. Chiang, *v4.brittanychiang.com — Personal Portfolio*, 2022. <https://v4.brittanychiang.com>
- [Goo24] Google, *Lighthouse Performance Scoring*, 2024. <https://developer.chrome.com/docs/lighthouse/performance/>
- [MDN24a] MDN Web Docs, *CSS Custom Properties (Variables)*, 2024. https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties
- [SO24] Stack Overflow, *Developer Survey 2024 — Most Popular Technologies*, 2024. <https://survey.stackoverflow.co/2024/>
- [GH24] GitHub, *Octoverse 2024 — State of Open Source*, 2024. <https://octoverse.github.com/>
- [Pup24] Puppeteer Team, *Puppeteer Documentation*, 2024. <https://pptr.dev/>
- [W3C23] W3C, *Web Content Accessibility Guidelines (WCAG) 2.2*, 2023. <https://www.w3.org/TR/WCAG22/>
- [Ver24b] Vercel, *Image Optimization in Next.js*, 2024. <https://nextjs.org/docs/app/building-your-application/optimizing/images>
- [Rea24] React Core Team, *React Server Components — RFC*, 2024. <https://github.com/reactjs/rfcs/blob/main/text/0188-server-components.md>
- [MDN24b] MDN Web Docs, *Intersection Observer API*, 2024. https://developer.mozilla.org/en-US/docs/Web/API/Intersection_Observer_API
- [EN07] R. Elmasri, S. Navathe, *Fundamentals of Database Systems*, 5th ed., Pearson/Addison Wesley, 2007.
- [Dil18] T. Dillahunt et al., *Digital Self-Presentation in Technical Hiring*, CHI 2018 Conference Proceedings, ACM, 2018.